

Kubernetes-04

Tasks:

1. Deploy an application using a Deployment with 3 replicas and a rolling update strategy.
2. Configure a Deployment with a Recreate strategy and observe the downtime.
3. Update an existing Deployment and perform a rollback to the previous version.
4. Modify a Deployment to add resource requests and limits for CPU and memory.
5. Create a Deployment with MaxSurge and MaxUnavailable configurations.
6. Set up a Deployment with a custom revision history limit.
7. Pause a Deployment during an update, and then resume it.
8. Create a pod using resource requests for memory and CPU, and observe how the scheduler assigns it to a node.

TASK 1 :Deploy an application using a Deployment with 3 replicas and a rolling update strategy.

Step 1 — Create new YAML

```
vi deploy1.yaml
```

```
apache-service.yaml createpod.yaml envpod.yaml
ubuntu@master:~$ vi deploy1.yaml
[ubuntu@master:~$ cat deploy1.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.24
        ports:
        - containerPort: 80

ubuntu@master:~$
```

Step 2 — Apply

```
kubectl apply -f deploy1.yaml
```

```
ubuntu@master:~$ kubectl apply -f deploy1.yaml
deployment.apps/web-deploy created
```

Step 3 — Verify the Deployment

Check if all 3 replicas are running:

```
kubectl get deployments
kubectl get pods
```

```
wait
[ubuntu@master:~]$ kubectl get deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
web-deploy    3/3     3            3           24s
[ubuntu@master:~]$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-deploy-7f859f48fb-fqd5b        1/1     Running   0          39s
web-deploy-7f859f48fb-gv9rv        1/1     Running   0          39s
web-deploy-7f859f48fb-j84n2        1/1     Running   0          39s
[ubuntu@master:~]$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
web-deploy    3/3     3            3           44s
ubuntu@master:~$
```

Step 4 — Watch Pods Live

Run this and keep it running:

```
kubectl get pods -w
```

```
Last login: Thu Feb 19 08:19:49 2026 from 49.43.226.202
ubuntu@master:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
web-deploy-7f859f48fb-fqd5b        1/1     Running   0          6m45s
web-deploy-7f859f48fb-gv9rv        1/1     Running   0          6m45s
web-deploy-7f859f48fb-j84n2        1/1     Running   0          6m45s
```

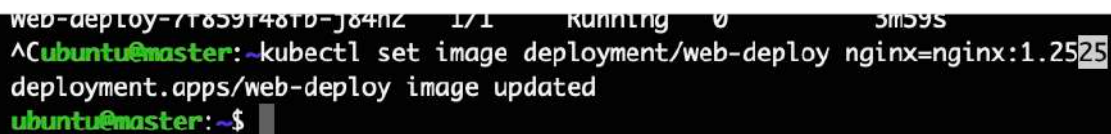
Don't stop it — this is like CCTV camera of Kubernetes

Open **another terminal** (or new tab SSH) and do next step.

Step 5 — Update Application Version

We change nginx version from 1.24 → 1.25

```
kubectl set image deployment/web-deploy nginx=nginx:1.25
```

A terminal window screenshot with a black background and white text. The top line shows pod status: 'web-deploy-7f859f481b-j84nz 1/1 Running 0 3m59s'. The second line shows a command being entered: '^Cubuntu@master:~\$ kubectl set image deployment/web-deploy nginx=nginx:1.25'. The third line shows the command being executed: 'deployment.apps/web-deploy image updated'. The fourth line shows the prompt again: 'ubuntu@master:~\$'.

```
web-deploy-7f859f481b-j84nz 1/1 Running 0 3m59s
^Cubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:1.25
deployment.apps/web-deploy image updated
ubuntu@master:~$
```

What You Should See

In the watch window:

Old pod dies → New pod comes → Next pod dies → Next pod comes

NOT all at once.

Example flow:

```
Terminating web-deploy-xxxxxx
Creating web-deploy-yyyyyy
Running web-deploy-yyyyyy
Terminating next old pod
Creating new pod
Running new pod
```

```

Last login: Thu Feb 19 08:19:49 2026 from 49.43.226.202
ubuntu@master:~$ kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
web-deploy-7f859f48fb-fqd5b        1/1     Running            0           6m45s
web-deploy-7f859f48fb-gv9rv        1/1     Running            0           6m45s
web-deploy-7f859f48fb-j84n2        1/1     Running            0           6m45s
web-deploy-686b75b84c-rjttx        0/1     Pending            0           0s
web-deploy-686b75b84c-rjttx        0/1     Pending            0           0s
web-deploy-686b75b84c-rjttx        0/1     ContainerCreating  0           0s
web-deploy-686b75b84c-rjttx        0/1     ContainerCreating  0           1s
web-deploy-686b75b84c-rjttx        1/1     Running            0           1s
web-deploy-7f859f48fb-gv9rv        1/1     Terminating      0           7m11s
web-deploy-686b75b84c-8l4qz        0/1     Pending            0           0s
web-deploy-686b75b84c-8l4qz        0/1     Pending            0           0s
web-deploy-686b75b84c-8l4qz        0/1     ContainerCreating  0           0s
web-deploy-7f859f48fb-gv9rv        1/1     Terminating      0           7m11s
web-deploy-7f859f48fb-gv9rv        0/1     Terminating      0           7m11s
web-deploy-686b75b84c-8l4qz        0/1     ContainerCreating  0           0s
web-deploy-7f859f48fb-gv9rv        0/1     Terminating      0           7m12s
web-deploy-7f859f48fb-gv9rv        0/1     Terminating      0           7m12s
web-deploy-686b75b84c-8l4qz        1/1     Running            0           4s
web-deploy-7f859f48fb-j84n2        1/1     Terminating      0           7m15s
web-deploy-686b75b84c-gd2t5        0/1     Pending            0           0s
web-deploy-686b75b84c-gd2t5        0/1     Pending            0           0s
web-deploy-686b75b84c-gd2t5        0/1     ContainerCreating  0           0s
web-deploy-7f859f48fb-j84n2        1/1     Terminating      0           7m15s
web-deploy-7f859f48fb-j84n2        0/1     Terminating      0           7m15s
web-deploy-686b75b84c-gd2t5        0/1     ContainerCreating  0           0s
web-deploy-686b75b84c-gd2t5        1/1     Running            0           1s
web-deploy-7f859f48fb-j84n2        0/1     Terminating      0           7m16s
web-deploy-7f859f48fb-j84n2        0/1     Terminating      0           7m16s
web-deploy-7f859f48fb-fqd5b        1/1     Terminating      0           7m16s
web-deploy-7f859f48fb-fqd5b        0/1     Terminating      0           7m17s
web-deploy-7f859f48fb-fqd5b        0/1     Terminating      0           7m17s
web-deploy-7f859f48fb-fqd5b        0/1     Terminating      0           7m17s

```

Users never lose service
 —> This is Rolling Update

Step 6 — Confirm Update Finished

```
kubectl rollout status deployment/web-deploy
```

```

deployment.apps/web-deploy image updated
ubuntu@master:~$ kubectl rollout status deployment/web-deploy
deployment "web-deploy" successfully rolled out
ubuntu@master:~$

```

Verify new ReplicaSet created

Run:

```
kubectl get rs
```

You will see **two ReplicaSets**

- old RS (0 pods)
- new RS (3 pods)

This proves rolling update happened.

```
deployment "web-deploy" successfully rolled out
ubuntu@master:~$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
web-deploy-686b75b84c              3         3         3       5m44s
web-deploy-7f859f48fb              0         0         0       12m
ubuntu@master:~$
```

```
web-deploy-686b75b84c-1jxkx-221-rolling-0-686b75b84c
ubuntu@master:~$ kubectl describe pod web-deploy-686b75b84c-814qz | grep Image
Image:          nginx:1.25
Image ID:       docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
Normal Pulled   8m47s kubelet      Successfully pulled image "nginx:1.25" in 2.667s (2.667s including waiting). Image size: 71005258 bytes.
ubuntu@master:~$
```

if You Want to See More Details About the Rolling Update

Run:

```
kubectl describe deployment web-deploy
```

```

ubuntu@master:~$ kubectl describe deployment web-deploy
Name: web-deploy
Namespace: default
CreationTimestamp: Thu, 19 Feb 2026 08:22:22 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 2
Selector: app=web
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=web
  Containers:
    nginx:
      Image: nginx:1.25
      Port: 80/TCP
      Host Port: 0/TCP
      Environment: <none>
      Mounts: <none>
  Volumes: <none>
  Node-Selectors: <none>
  Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: web-deploy-7f859f48fb (0/0 replicas created)
NewReplicaSet:  web-deploy-686b75b84c (3/3 replicas created)
Events:
  Type     Reason             Age   From                    Message
  ----     -
  Normal   ScalingReplicaSet   17m   deployment-controller   Scaled up replica set web-deploy-7f859f48fb to 3
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled up replica set web-deploy-686b75b84c to 1
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled down replica set web-deploy-7f859f48fb to 2 from 3
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled up replica set web-deploy-686b75b84c to 2 from 1
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled down replica set web-deploy-7f859f48fb to 1 from 2
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled up replica set web-deploy-686b75b84c to 3 from 2
  Normal   ScalingReplicaSet   10m   deployment-controller   Scaled down replica set web-deploy-7f859f48fb to 0 from 1
ubuntu@master:~$

```

TASK-2 — Configure a Deployment with a Recreate strategy and observe the downtime.

—> Recreate Strategy (Downtime Demo)

Step 1 — Edit Deployment YAML

Open file:

```
vi deploy1.yaml
```

Change strategy section :

From:

```

strategy:
  type: RollingUpdate

```

To:

```
strategy:  
  type: Recreate
```

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: web-deploy  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: web  
  strategy:  
    type: Recreate  
  template:  
    metadata:  
      labels:  
        app: web  
    spec:  
      containers:  
        - name: nginx  
          image: nginx:1.24  
          ports:  
            - containerPort: 80
```

```
~  
~  
~  
~
```

—>save exit

Step 2 — Apply the Change

```
kubectl apply -f deploy1.yaml
```

This only changes strategy — pods won't restart yet.

```
ubuntu@master:~$ vi deploy1.yaml
ubuntu@master:~$ kubectl apply -f deploy1.yaml
deployment.apps/web-deploy configured
ubuntu@master:~$
```

Step 3 — Watch Pods (VERY IMPORTANT)

Start watching:

```
kubectl get pods -w
```

Keep this running like CCTV camera

```
^Cubuntu@master:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
web-deploy-686b75b84c-8l4qz         1/1     Running   0           17m
web-deploy-686b75b84c-gd2t5         1/1     Running   0           17m
web-deploy-686b75b84c-rjtxx         1/1     Running   0           17m
web-deploy-686b75b84c-8l4qz         1/1     Terminating   0           17m
web-deploy-686b75b84c-rjtxx         1/1     Terminating   0           17m
web-deploy-686b75b84c-gd2t5         1/1     Terminating   0           17m
web-deploy-686b75b84c-8l4qz         1/1     Terminating   0           17m
web-deploy-686b75b84c-gd2t5         1/1     Terminating   0           17m
web-deploy-686b75b84c-rjtxx         1/1     Terminating   0           17m
web-deploy-686b75b84c-8l4qz         0/1     Terminating   0           17m
web-deploy-686b75b84c-gd2t5         0/1     Terminating   0           17m
web-deploy-686b75b84c-rjtxx         0/1     Terminating   0           17m
web-deploy-7f859f48fb-kgw24         0/1     Pending      0           0s
web-deploy-7f859f48fb-dtfwj         0/1     Pending      0           0s
web-deploy-7f859f48fb-kgw24         0/1     Pending      0           0s
web-deploy-7f859f48fb-dpn67         0/1     Pending      0           0s
web-deploy-7f859f48fb-dtfwj         0/1     Pending      0           0s
web-deploy-7f859f48fb-dpn67         0/1     Pending      0           0s
web-deploy-7f859f48fb-kgw24         0/1     ContainerCreating   0           0s
web-deploy-7f859f48fb-dtfwj         0/1     ContainerCreating   0           0s
web-deploy-7f859f48fb-dpn67         0/1     ContainerCreating   0           0s
web-deploy-686b75b84c-rjtxx         0/1     Terminating   0           17m
web-deploy-686b75b84c-rjtxx         0/1     Terminating   0           17m
web-deploy-686b75b84c-gd2t5         0/1     Terminating   0           17m
web-deploy-686b75b84c-gd2t5         0/1     Terminating   0           17m
web-deploy-7f859f48fb-dtfwj         0/1     ContainerCreating   0           0s
web-deploy-7f859f48fb-kgw24         0/1     ContainerCreating   0           0s
web-deploy-7f859f48fb-dpn67         0/1     ContainerCreating   0           0s
web-deploy-7f859f48fb-dtfwj         1/1     Running        0           0s
web-deploy-686b75b84c-8l4qz         0/1     Terminating   0           17m
web-deploy-686b75b84c-8l4qz         0/1     Terminating   0           17m
web-deploy-7f859f48fb-kgw24         1/1     Running        0           1s
web-deploy-7f859f48fb-dpn67         1/1     Running        0           1s
```

i-0c3faabb470a01398 (k8s-master)

Step 4: Confirm Strategy Type

To verify the strategy used:

```
kubectl describe deployment web-deploy | grep StrategyType
```

```
deployment.apps/web-deploy configured
[ubuntu@master:~$ kubectl describe deployment web-deploy | grep StrategyType
StrategyType:      Recreate
ubuntu@master:~$
```

Step 5 — Trigger Update

Open second terminal and run:

```
kubectl set image deployment/web-deploy nginx=nginx:alpine
```

```
deployment.apps/web-deploy image updated
[ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:alpine
deployment.apps/web-deploy image updated
ubuntu@master:~$
```

Step 4 — Start Watching in other terminal

Run:

```
kubectl get pods -w
Keep this open.
```

```
ubuntu@master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
web-deploy-686b75b84c-2rwgk	1/1	Running	0	44s
web-deploy-686b75b84c-nlprg	1/1	Running	0	44s
web-deploy-686b75b84c-vrd6h	1/1	Running	0	44s
web-deploy-686b75b84c-nlprg	1/1	Terminating	0	48s
web-deploy-686b75b84c-vrd6h	1/1	Terminating	0	48s
web-deploy-686b75b84c-2rwgk	1/1	Terminating	0	48s
web-deploy-686b75b84c-nlprg	1/1	Terminating	0	49s
web-deploy-686b75b84c-vrd6h	1/1	Terminating	0	49s
web-deploy-686b75b84c-2rwgk	1/1	Terminating	0	49s
web-deploy-686b75b84c-nlprg	0/1	Terminating	0	49s
web-deploy-686b75b84c-vrd6h	0/1	Terminating	0	49s
web-deploy-686b75b84c-2rwgk	0/1	Terminating	0	49s
web-deploy-686b75b84c-nlprg	0/1	Terminating	0	49s
web-deploy-686b75b84c-nlprg	0/1	Terminating	0	49s
web-deploy-686b75b84c-2rwgk	0/1	Terminating	0	50s
web-deploy-686b75b84c-2rwgk	0/1	Terminating	0	50s
web-deploy-686b75b84c-vrd6h	0/1	Terminating	0	50s
web-deploy-686b75b84c-vrd6h	0/1	Terminating	0	50s
web-deploy-85b78f8978-x9t72	0/1	Pending	0	0s
web-deploy-85b78f8978-fg24z	0/1	Pending	0	0s
web-deploy-85b78f8978-x9t72	0/1	Pending	0	0s
web-deploy-85b78f8978-59kq4	0/1	Pending	0	0s
web-deploy-85b78f8978-fg24z	0/1	Pending	0	0s
web-deploy-85b78f8978-59kq4	0/1	Pending	0	0s
web-deploy-85b78f8978-x9t72	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-fg24z	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-59kq4	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-x9t72	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-fg24z	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-59kq4	0/1	ContainerCreating	0	0s
web-deploy-85b78f8978-x9t72	1/1	Running	0	0s
web-deploy-85b78f8978-59kq4	1/1	Running	0	1s
web-deploy-85b78f8978-fg24z	1/1	Running	0	1s

Observation

- All old pods went into **Terminating** state simultaneously.
- For a few seconds, **0 pods were running**.
- New pods entered **Pending** → **ContainerCreating** → **Running** state.
- This confirmed service unavailability during update.

Conclusion

Recreate strategy deletes all existing pods before creating new ones, causing temporary downtime. It is suitable only when old and new versions cannot run together.

Task-3: Update Deployment & Rollback

3.Update an existing Deployment and perform a rollback to the previous version.

Step 1 — Check current deployment

```
kubectl get deployments
```

```
ubuntu@master:~$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
web-deploy    3/3     3            3           48m
ubuntu@master:~$
```

Step 2 — Check current image version

```
kubectl describe deployment web-deploy | grep Image
```

You will see something like:

```
Image: nginx:alpine
```

```
web-deploy 3/3     3            3           48m
ubuntu@master:~$ kubectl describe deployment web-deploy | grep Image
Image:      nginx:alpine
ubuntu@master:~$
```

Step 3 — Update to a new version

(Update to any valid version)

```
kubectl set image deployment/web-deploy nginx=nginx:1.27.0
```

Check rollout:

```
kubectl rollout status deployment/web-deploy
```

```
ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:1.27.0
deployment.apps/web-deploy image updated
ubuntu@master:~$ kubectl rollout status deployment/web-deploy
deployment "web-deploy" successfully rolled out
ubuntu@master:~$
```

Step 4 — Verify update happened

```
kubectl get pods -w
```

You should see old pods terminating and new pods creating.

```
ubuntu@master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
web-deploy-85b78f8978-59kq4	1/1	Running	0	14m
web-deploy-85b78f8978-fg24z	1/1	Running	0	14m
web-deploy-85b78f8978-x9t72	1/1	Running	0	14m
web-deploy-85b78f8978-59kq4	1/1	Terminating	0	16m
web-deploy-85b78f8978-x9t72	1/1	Terminating	0	16m
web-deploy-85b78f8978-fg24z	1/1	Terminating	0	16m
web-deploy-85b78f8978-x9t72	1/1	Terminating	0	16m
web-deploy-85b78f8978-x9t72	0/1	Terminating	0	16m
web-deploy-85b78f8978-59kq4	1/1	Terminating	0	16m
web-deploy-85b78f8978-fg24z	1/1	Terminating	0	16m
web-deploy-85b78f8978-fg24z	0/1	Terminating	0	16m
web-deploy-85b78f8978-59kq4	0/1	Terminating	0	16m
web-deploy-5f46bd5859-k521s	0/1	Pending	0	0s
web-deploy-5f46bd5859-k521s	0/1	Pending	0	0s
web-deploy-5f46bd5859-wvc5h	0/1	Pending	0	0s
web-deploy-5f46bd5859-n2ssg	0/1	Pending	0	0s
web-deploy-5f46bd5859-k521s	0/1	ContainerCreating	0	0s
web-deploy-5f46bd5859-wvc5h	0/1	Pending	0	0s
web-deploy-5f46bd5859-n2ssg	0/1	Pending	0	0s
web-deploy-5f46bd5859-wvc5h	0/1	ContainerCreating	0	0s
web-deploy-5f46bd5859-n2ssg	0/1	ContainerCreating	0	0s
web-deploy-5f46bd5859-k521s	0/1	ContainerCreating	0	1s
web-deploy-5f46bd5859-n2ssg	0/1	ContainerCreating	0	1s
web-deploy-5f46bd5859-wvc5h	0/1	ContainerCreating	0	1s
web-deploy-85b78f8978-x9t72	0/1	Terminating	0	16m
web-deploy-85b78f8978-x9t72	0/1	Terminating	0	16m
web-deploy-85b78f8978-fg24z	0/1	Terminating	0	16m
web-deploy-85b78f8978-fg24z	0/1	Terminating	0	16m
web-deploy-85b78f8978-59kq4	0/1	Terminating	0	16m
web-deploy-85b78f8978-59kq4	0/1	Terminating	0	16m
web-deploy-5f46bd5859-k521s	1/1	Running	0	4s
web-deploy-5f46bd5859-wvc5h	1/1	Running	0	4s
web-deploy-5f46bd5859-n2ssg	1/1	Running	0	4s

Confirm image:

```
kubectl describe deployment web-deploy | grep Image
```

```
ubuntu@master:~$ kubectl describe deployment web-deploy | grep Image
Image:          nginx:1.27.0
ubuntu@master:~$
```

Step 5 — Now simulate failure (important for rollback)

```
kubectl set image deployment/web-deploy
nginx=nginx:wrongversion
```

```
ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:wrongversion
deployment.apps/web-deploy image updated
ubuntu@master:~$
```

```
ubuntu@master:~$ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
web-deploy-7cc8c4c9b6-kfkr1        0/1     ImagePullBackOff    0           19s
web-deploy-7cc8c4c9b6-l4s67        0/1     ImagePullBackOff    0           19s
web-deploy-7cc8c4c9b6-znf88        0/1     ImagePullBackOff    0           19s
ubuntu@master:~$
```

i-0c3faabb470a01398 (k8s-master)

Your deployment is now **broken**:

ImagePullBackOff

Meaning:

Kubernetes tried to pull image → failed → pods not running → app down

Now Perform the Rollback

Step 6 — See versions

```
kubectl rollout history deployment/web-deploy
```

```
last login: Thu Feb 15 08:28:43 EST 2018 from 57.126.218.1  
[ubuntu@master:~]$ kubectl rollout history deployment/web-deploy  
deployment.apps/web-deploy  
REVISION  CHANGE-CAUSE  
3          <none>  
5          <none>  
6          <none>  
7          <none>  
8          <none>  
  
ubuntu@master:~$
```

Step 7 — Rollback to previous working version

```
kubectl rollout undo deployment/web-deploy
```

```
ubuntu@master:~$ kubectl rollout undo deployment/web-deploy  
  
deployment.apps/web-deploy rolled back  
ubuntu@master:~$ kubectl rollout undo deployment/web-deploy
```

Step 8 — Watch recovery

```
kubectl rollout status deployment/web-deploy  
kubectl get pods
```

```
ubuntu@master:~$ kubectl rollout status deployment/web-deploy  
deployment "web-deploy" successfully rolled out  
ubuntu@master:~$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
web-deploy-5f46bd5859-2hbfd         1/1     Running   0           39s  
web-deploy-5f46bd5859-48qrf         1/1     Running   0           39s  
web-deploy-5f46bd5859-njb56         1/1     Running   0           39s  
ubuntu@master:~$ \
```

i-0c3faabb470a01398 (k8s-master)

PublicIP: 3.135.182.25 PrivateIP: 172.31.14.17

—> successfully rollbacked to previous version of nginx

```
ubuntu@master:~$ kubectl describe deployment web-deploy | grep Image  
Image:          nginx:1.27.0  
ubuntu@master:~$
```

Task-4:

Modify a Deployment to add resource requests and limits for CPU and memory.

Objective

Configure a Kubernetes Deployment so the container uses defined **minimum** and **maximum** CPU & Memory.

Step 1 — Edit the Deployment YAML

Open your deployment file:

```
vi deploy1.yaml
```

Add the `resources` section inside the container:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "200m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

—> save exit

Step 2 — Apply the configuration

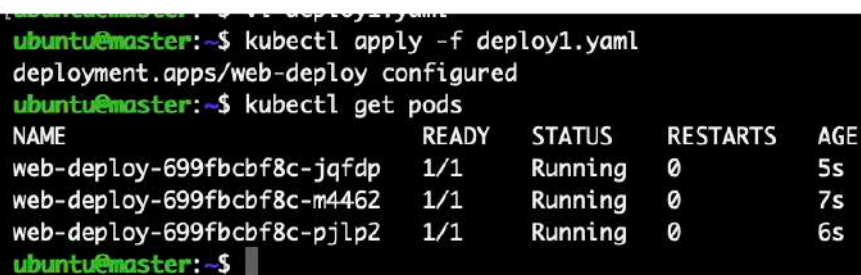
```
kubectl apply -f deploy1.yaml
```

Pods will restart and a new revision will be created.

Step 3 — Verify resources

Get pod name:

```
kubectl get pods
```



```
ubuntu@master:~$ kubectl apply -f deploy1.yaml
deployment.apps/web-deploy configured
ubuntu@master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-deploy-699fbcbf8c-jqfdp        1/1     Running   0           5s
web-deploy-699fbcbf8c-m4462        1/1     Running   0           7s
web-deploy-699fbcbf8c-pjlp2        1/1     Running   0           6s
ubuntu@master:~$
```

Check details:

```
kubectl describe pod <pod-name>
```

You should see:

Requests:

```
cpu:    200m
memory: 128Mi
```

Limits:

```
cpu:    500m
memory: 256Mi
```

```

[ubuntu@master:~]$ kubectl describe pod web-deploy-699fbcfb8c-jqfdp
Name:          web-deploy-699fbcfb8c-jqfdp
Namespace:     default
Priority:       0
Service Account: default
Node:          worker2/172.31.9.239
Start Time:    Thu, 19 Feb 2026 09:38:12 +0000
Labels:        app=web
Annotations:   pod-template-hash=699fbcfb8c
               cni.projectcalico.org/containerID: 2a70ae894ea0da9e346ad1932b9dea4e13eda6e6417b4d6305064303ca9760ce
               cni.projectcalico.org/podIP: 192.168.189.102/32
               cni.projectcalico.org/podIPs: 192.168.189.102/32
Status:        Running
IP:            192.168.189.102
IPs:           192.168.189.102
Controlled By: ReplicaSet/web-deploy-699fbcfb8c
Containers:
  nginx:
    Container ID:   containerd://d0a9e4b903b9a3078ce766d8f9095c12be42e32732f5f4522ac1851ea29ed22c
    Image:          nginx:1.25
    Image ID:       docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Thu, 19 Feb 2026 09:38:13 +0000
    Ready:          True
    Restart Count:  0
    Limits:         cpu: 500m
                   memory: 256Mi
    Requests:       cpu: 200m
                   memory: 128Mi
    Environment:    <none>
    Mounts:          /var/run/cacerts/kubernetes.io/serviceaccount from kube-api-access-sgqhj (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                         True
  Ready                              True
  ContainersReady                    True
  PodScheduled                       True
Volumes:
  kube-api-access-sgqhj:
    Type:  Projected (a volume that contains injected data from multiple sources)

```

Observation

- Scheduler placed pods based on requested resources.
- Container cannot exceed defined limits.

Conclusion

Resource requests ensure guaranteed minimum resources for scheduling, while limits restrict the container from over-consuming CPU and memory.

Task-5: Create a Deployment with MaxSurge and MaxUnavailable configurations

Objective

Configure a rolling update strategy to control how many pods are created and terminated during an update.

Step 1 — Edit the Deployment YAML

Open the file:

```
vi deploy1.yaml
```

Add the rolling update configuration inside `spec`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1

  selector:
    matchLabels:
      app: web

  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

First terminal → start watching

Open terminal-1 and run:

```
kubectl get pods -w
```

Keep it running.

Step 2 — Apply configuration

```
kubectl apply -f deploy1.yaml
```

```
[ubuntu@master:~$ kubectl apply -f deploy1.yaml  
deployment.apps/web-deploy created
```

You will immediately see pods appearing in the watch window.

Wait until **3 pods become Running**

```
Last login: Thu Feb 19 09:36:31 2026 from 49.43.226.202  
ubuntu@master:~$ kubectl get pods -w  
NAME                                READY   STATUS             RESTARTS   AGE  
web-deploy-686b75b84c-npd8x         0/1     Pending            0           0s  
web-deploy-686b75b84c-6rkn2         0/1     Pending            0           0s  
web-deploy-686b75b84c-7t6rd         0/1     Pending            0           0s  
web-deploy-686b75b84c-npd8x         0/1     Pending            0           0s  
web-deploy-686b75b84c-6rkn2         0/1     Pending            0           0s  
web-deploy-686b75b84c-7t6rd         0/1     Pending            0           0s  
web-deploy-686b75b84c-npd8x         0/1     ContainerCreating  0           0s  
web-deploy-686b75b84c-6rkn2         0/1     ContainerCreating  0           0s  
web-deploy-686b75b84c-7t6rd         0/1     ContainerCreating  0           0s  
web-deploy-686b75b84c-6rkn2         0/1     ContainerCreating  0           1s  
web-deploy-686b75b84c-npd8x         0/1     ContainerCreating  0           1s  
web-deploy-686b75b84c-7t6rd         0/1     ContainerCreating  0           1s  
web-deploy-686b75b84c-6rkn2         1/1     Running            0           1s  
web-deploy-686b75b84c-7t6rd         1/1     Running            0           1s  
web-deploy-686b75b84c-npd8x         1/1     Running            0           2s
```

```
i-0c3faabb470a01398 (k8s-master)
```

Step:3

Trigger rolling update (while still watching)

In the second terminal run:

```
kubectl set image deployment/web-deploy nginx=nginx:1.26
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The command 'kubectl set image deployment/web-deploy nginx=nginx:1.26' is entered. The output is 'deployment.apps/web-deploy image updated'. The prompt is then 'ubuntu@master:~\$' followed by a cursor.

```
ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:1.26
deployment.apps/web-deploy image updated
ubuntu@master:~$
```

What you should observe

Because:

```
replicas = 3
maxSurge = 1
maxUnavailable = 1
```

```

web-deploy-558f486f9d-g5g9b 0/1 Pending 0 0s
web-deploy-558f486f9d-g5g9b 0/1 Pending 0 0s
web-deploy-686b75b84c-6rkn2 1/1 Terminating 0 77s
web-deploy-558f486f9d-g5g9b 0/1 ContainerCreating 0 0s
web-deploy-558f486f9d-jdrbv 0/1 Pending 0 0s
web-deploy-558f486f9d-jdrbv 0/1 Pending 0 0s
web-deploy-558f486f9d-jdrbv 0/1 ContainerCreating 0 0s
web-deploy-686b75b84c-6rkn2 1/1 Terminating 0 77s
web-deploy-686b75b84c-6rkn2 0/1 Terminating 0 77s
web-deploy-558f486f9d-g5g9b 0/1 ContainerCreating 0 0s
web-deploy-558f486f9d-jdrbv 0/1 ContainerCreating 0 0s
web-deploy-558f486f9d-jdrbv 1/1 Running 0 0s
web-deploy-686b75b84c-6rkn2 0/1 Terminating 0 77s
web-deploy-686b75b84c-7t6rd 1/1 Terminating 0 77s
web-deploy-686b75b84c-6rkn2 0/1 Terminating 0 77s
web-deploy-558f486f9d-tzlwz 0/1 Pending 0 0s
web-deploy-558f486f9d-tzlwz 0/1 Pending 0 0s
web-deploy-558f486f9d-tzlwz 0/1 ContainerCreating 0 0s
web-deploy-686b75b84c-7t6rd 1/1 Terminating 0 78s
web-deploy-686b75b84c-7t6rd 0/1 Terminating 0 78s
web-deploy-558f486f9d-g5g9b 1/1 Running 0 1s
web-deploy-686b75b84c-npd8x 1/1 Terminating 0 78s
web-deploy-686b75b84c-npd8x 1/1 Terminating 0 78s
web-deploy-558f486f9d-tzlwz 0/1 ContainerCreating 0 1s
web-deploy-686b75b84c-npd8x 0/1 Terminating 0 78s
web-deploy-558f486f9d-tzlwz 1/1 Running 0 1s
web-deploy-686b75b84c-7t6rd 0/1 Terminating 0 78s
web-deploy-686b75b84c-7t6rd 0/1 Terminating 0 78s
web-deploy-686b75b84c-npd8x 0/1 Terminating 0 79s
web-deploy-686b75b84c-npd8x 0/1 Terminating 0 79s

```

i-0c3faabb470a01398 (k8s-master)

PublicIPs: 3.135.182.35 PrivateIPs: 172.31.14.17

```

[ubuntu@master:~]$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
web-deploy-558f486f9d-g5g9b        1/1     Running   0           3m5s
web-deploy-558f486f9d-jdrbv        1/1     Running   0           3m5s
web-deploy-558f486f9d-tzlwz        1/1     Running   0           3m5s
ubuntu@master:~$

```

Let me translate what happened in simple words.

You had:

replicas = 3

maxSurge = 1

maxUnavailable = 1

So Kubernetes rule became:

- Minimum running pods = 2
- Maximum running pods = 4

What your log shows

You saw lines like:

```
old pod Terminating
new pod Pending → ContainerCreating → Running
another old pod Terminating
another new pod Creating → Running
Very important :
Kubernetes did NOT delete all old pods first
```

Instead it followed this pattern:

1. Create a new pod
2. Then remove one old pod
3. Again create a new pod
4. Then remove one old pod

So at every moment some pods were still running.

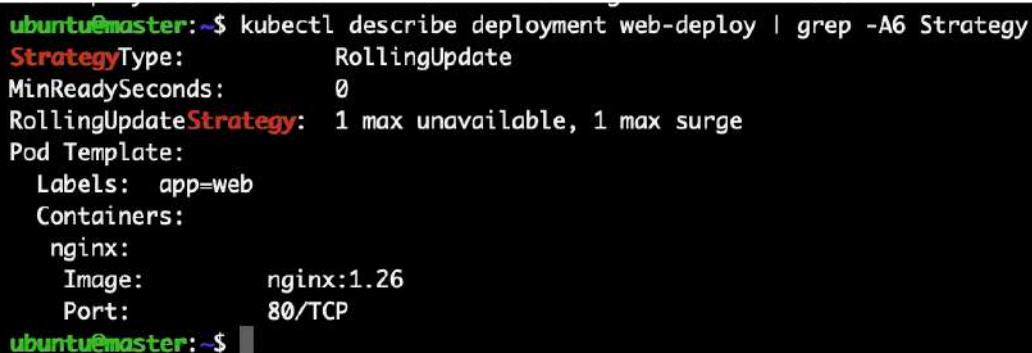
That means:

- >Application never stopped
- >Controlled rolling update happened
- >Strategy settings worked correctly

Verify RollingUpdate Strategy

Run:

```
kubectl describe deployment web-deploy | grep -A6 Strategy
```



```
ubuntu@master:~$ kubectl describe deployment web-deploy | grep -A6 Strategy
StrategyType:          RollingUpdate
MinReadySeconds:       0
RollingUpdateStrategy: 1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=web
  Containers:
    nginx:
      Image:          nginx:1.26
      Port:           80/TCP
ubuntu@master:~$
```

Check ReplicaSets created

`kubectl get rs`

You should see:

- old RS scaling down
- new RS scaling up

✓ proves rolling mechanism

Confirm rollout finished

`kubectl rollout status deployment/web-deploy`

✓ proves update completed successfully



```
ubuntu@master:~$ kubectl get rs
NAME                                DESIRED   CURRENT   READY   AGE
web-deploy-558f486f9d              3         3         3       9m7s
web-deploy-686b75b84c              0         0         0       10m
ubuntu@master:~$
ubuntu@master:~$ kubectl rollout status deployment/web-deploy
deployment "web-deploy" successfully rolled out
ubuntu@master:~$
```

Task-6.Set up a Deployment with a custom revision history limit.

Objective

Limit how many old versions (ReplicaSets) Kubernetes stores for a Deployment.

Default → keeps **10 old versions**

We will change → keep only **2 old versions**

Step 1 — Edit Deployment YAML

Open your file:

```
vi deploy1.yaml
```

Add this line under **spec:** (top level inside spec)

```
revisionHistoryLimit: 2
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3
  revisionHistoryLimit: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

Step 2 — Apply Changes

```
kubectl apply -f deploy1.yaml
```

```
ubuntu@master:~$ kubectl apply -f deploy1.yaml
deployment.apps/web-deploy configured
ubuntu@master:~$ kubectl get rs
NAME                                DESIRED    CURRENT    READY    AGE
web-deploy-558f486f9d              0          0          0        19m
web-deploy-686b75b84c              3          3          3        20m
```

Step 3 — Create multiple revisions (important)

Run these one by one:

```
kubectl set image deployment/web-deploy nginx=nginx:1.26
kubectl set image deployment/web-deploy nginx=nginx:1.27
kubectl set image deployment/web-deploy nginx=nginx:1.24
kubectl set image deployment/web-deploy nginx=nginx:1.25
Each command = new version
```

```
web-deploy-686b75b84c              3          3          3        20m
ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:1.26
kubectl set image deployment/web-deploy nginx=nginx:1.27
kubectl set image deployment/web-deploy nginx=nginx:1.24
kubectl set image deployment/web-deploy nginx=nginx:1.25
deployment.apps/web-deploy image updated
deployment.apps/web-deploy image updated
deployment.apps/web-deploy image updated
deployment.apps/web-deploy image updated
```

What happened internally

Each command created:

- New ReplicaSet
- New Deployment revision

So Kubernetes generated many versions of the same application.

Checked ReplicaSets again

```
kubectl get rs
```

Output:

```
web-deploy-686b75b84c    3 3 3    ← current
web-deploy-7f859f48fb    0 0 0
web-deploy-c54748bb5     0 0 0
```

```
deployment.apps/web-deploy image updated
[ubuntu@master:~]$ kubectl get rs
NAME                                DESIRED    CURRENT    READY    AGE
web-deploy-686b75b84c              3          3          3        21m
web-deploy-7f859f48fb              0          0          0        10s
web-deploy-c54748bb5               0          0          0        10s
```

Important Observation

Only **3 ReplicaSets** exist

- 1 active
- 2 previous

All older ReplicaSets automatically deleted

This proves `revisionHistoryLimit: 2` worked

Verified rollout history

```
kubectl rollout history deployment/web-deploy
```

```

web-deploy-7f859f48fb 0 0 0 10s
web-deploy-c54748bb5 0 0 0 10s
ubuntu@master:~$ kubectl rollout history deployment/web-deploy
deployment.apps/web-deploy
REVISION  CHANGE-CAUSE
5          <none>
6          <none>
7          <none>

ubuntu@master:~$ Read from remote host ec2-3-135-182-35.us-east-2.compute.amazonaws.com

```

Explanation

Earlier revisions (1,2,3,4) disappeared
Kubernetes kept only recent revisions

Final Observation

Even though many updates were performed, Kubernetes retained only the latest two old versions and removed earlier ones automatically.

Conclusion

`revisionHistoryLimit` controls how many previous versions Kubernetes stores.
It prevents excessive storage usage and keeps limited rollback capability.

```

ubuntu@master:~$ kubectl get deployment web-deploy -o yaml | grep -A1 revisionHistoryLimit
{"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"name":"web-deploy","namespace":"default"},"spec":{"replicas":3,"revisionHistoryLimit":2,
llingUpdate"},"template":{"metadata":{"labels":{"app":"web"},"spec":{"containers":[{"image":"nginx:1.25","name":"nginx","ports":[{"containerPort":80}]}]}}}
creationTimestamp: "2026-02-19T09:49:11Z"
--
revisionHistoryLimit: 2
selector:
ubuntu@master:~$

```

Task-7: Pause a Deployment and Resume it

Purpose:

Stop rollout temporarily → make changes → then continue rollout

Very useful in production when you want to **review changes before applying**

Step 1 — Pause the Deployment

```
kubectl rollout pause deployment/web-deploy
```

Check status:

```
selector:  
[ubuntu@master:~]$ kubectl rollout pause deployment/web-deploy  
deployment.apps/web-deploy paused  
ubuntu@master:~$
```

```
kubectl describe deployment web-deploy | grep -i paused
```

Expected:

Paused: True

```
ubuntu@master:~$ kubectl describe deployment web-deploy | grep -i paused  
Progressing   Unknown   DeploymentPaused  
ubuntu@master:~$
```

Step 2 — Update the image (nothing should happen yet)

```
kubectl set image deployment/web-deploy nginx=nginx:1.26
```

Now check pods:

```
kubectl get pods
```

```
ubuntu@master:~$ kubectl set image deployment/web-deploy nginx=nginx:1.26
deployment.apps/web-deploy image updated
ubuntu@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-deploy-686b75b84c-fndvd	1/1	Running	0	25m
web-deploy-686b75b84c-ggwk4	1/1	Running	0	25m
web-deploy-686b75b84c-njnfv	1/1	Running	0	25m

```
ubuntu@master:~$
```

Observation

Pods will NOT change

No rollout starts because deployment is paused

Step 3 — Resume the Deployment

```
kubectl rollout resume deployment/web-deploy
```

```
ubuntu@master:~$ kubectl rollout resume deployment/web-deploy
deployment.apps/web-deploy resumed
ubuntu@master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
web-deploy-558f486f9d-jp7rw	1/1	Running	0	10s
web-deploy-558f486f9d-rmnfm	1/1	Running	0	11s
web-deploy-558f486f9d-sx966	1/1	Running	0	11s

Step 4 — Watch rollout start

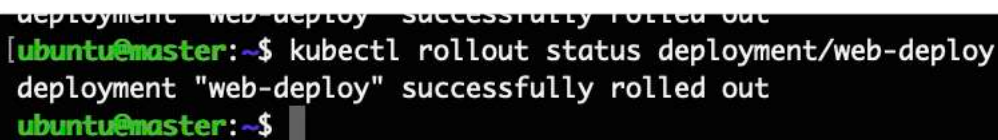
```
kubectl get pods -w
```

Now you will see:

Old pods terminate → new pods create

Step 5 — Verify completion

```
kubectl rollout status deployment/web-deploy
```

A terminal window with a black background and green text. The first line shows a partial command: 'deployment web-deploy successfully rolled out'. The second line shows the full command: '[ubuntu@master:~\$ kubectl rollout status deployment/web-deploy'. The third line shows the output: 'deployment "web-deploy" successfully rolled out'. The fourth line shows the prompt: 'ubuntu@master:~\$' followed by a cursor.

```
deployment web-deploy successfully rolled out  
[ubuntu@master:~$ kubectl rollout status deployment/web-deploy  
deployment "web-deploy" successfully rolled out  
ubuntu@master:~$
```

What we proved

- Pause stopped update
- Changes were stored but not applied
- Resume triggered rollout

Conclusion

Pausing a deployment temporarily stops rollout allowing safe modifications, and resuming continues the update process.

Task-8 — Create a Pod with resource requests & observe scheduler node selection

Goal:

Show that Kubernetes places a pod on a node **based on available CPU & Memory**

Step 1 — Create Pod YAML

Create file:

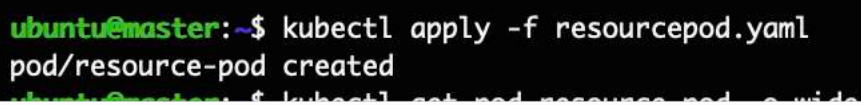
```
vi resourcepod.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi resourcepod.yaml'. The prompt changes to '[ubuntu@master:~\$ cat resourcepod.yaml'. The content of the file is displayed: 'apiVersion: v1', 'kind: Pod', 'metadata:', ' name: resource-pod', 'spec:', ' containers:', ' - name: nginx', ' image: nginx', ' resources:', ' requests:', ' cpu: "500m"', ' memory: "256Mi"', and 'ubuntu@master:~\$' with a cursor.

```
Normal ScalingReplicaSet 2m45s (x2 over 48m) deployment
ubuntu@master:~$ vi resourcepod.yaml
[ubuntu@master:~$ cat resourcepod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
ubuntu@master:~$
```

Step 2 — Apply Pod

```
kubectl apply -f resourcepod.yaml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'kubectl apply -f resourcepod.yaml'. The output is 'pod/resource-pod created'. The prompt changes to 'ubuntu@master:~\$' and the user enters 'kubectl get pod resource-pod -o wide'.

```
ubuntu@master:~$ kubectl apply -f resourcepod.yaml
pod/resource-pod created
ubuntu@master:~$ kubectl get pod resource-pod -o wide
```

Step 3 — Check which node it was scheduled to

```
kubectl get pod resource-pod -o wide
```

```
ubuntu@master:~$ kubectl apply -f resourcepod.yaml
pod/resource-pod created
ubuntu@master:~$ kubectl get pod resource-pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
resource-pod	1/1	Running	0	7s	192.168.189.114	worker2	<none>	<none>

```
ubuntu@master:~$ kubectl describe pod resource-pod
```

Scheduler selected node that had enough free resources

Step 4 — Verify from describe

```
kubectl describe pod resource-pod
```

```
ubuntu@master:~$ kubectl describe pod resource-pod
Name:          resource-pod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker2/172.31.9.239
Start Time:    Thu, 19 Feb 2026 10:43:34 +0000
Labels:        <none>
Annotations:    cni.projectcalico.org/containerID: 3e87ae2b9146ecdec294cb37700d6f40be893477601b3ade77d002e0688bb2d
                 cni.projectcalico.org/podIP: 192.168.189.114/32
                 cni.projectcalico.org/podIPs: 192.168.189.114/32
Status:        Running
IP:            192.168.189.114
IPs:
  IP: 192.168.189.114
Containers:
  nginx:
    Container ID:  containerd://42b706a16948a9189de938513bcd8a469fb7dce0587ae58d81728cd41a23597
    Image:          nginx
    Image ID:       docker.io/library/nginx:sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Thu, 19 Feb 2026 10:43:35 +0000
    Ready:         True
    Restart Count: 0
    Requests:
      cpu:          500m
      memory:       256Mi
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-kc6mr (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers         True
```

Observation

Pod was assigned to a node that satisfied requested CPU & Memory.

```
Normal Started 15s kubelet Started container nginx
[ubuntu@master:~$ curl 192.168.189.114
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@master:~$
```

Conclusion

Kubernetes scheduler places pods on nodes based on resource requests to ensure efficient resource utilization.

